



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423603
(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 2001:2018 Certified
Department of Information Technology
NBA Accredited-UG Programme

Class: B. Tech. Semester: VII

Subject: Advanced Data Science Laboratory (PEIT412B)

Practical No.: 01

CO1: Implement deep learning models using TensorFlow/PyTorch frameworks. (BTL-Apply)

Title:

Assignment based on implementation of Artificial Neural Network (ANN) using TensorFlow/Keras for prediction problems.

Software Requirements:

- Python (Recent Version), TensorFlow/Keras, Pandas, Scikit-learn.
- Visual Studio Code (or any other editor), Jupyter Notebook / Google Colab.

Learning Outcomes:

By the end of this assignment, students should be able to:

- Understand the basic architecture and working of an Artificial Neural Network (ANN).
- Build, compile, and train ANN models using TensorFlow/Keras.
- Apply ANN models to solve both regression and classification prediction problems.
- Understand the role of activation functions, loss functions, and optimizers.
- Evaluate ANN model performance using appropriate metrics (MAE, RMSE, Accuracy, F1-Score).

Exemplars / Case Study:

Implement ANN models for the following prediction problems:

- Student Performance Prediction: Predict a student's marks/grade (regression) using features such as study hours, attendance, and past scores.
- House Price Prediction: Predict house prices (regression) based on features like area, location, number of rooms, and age of property.
- Employee Attrition Prediction: Predict whether an employee will leave the company (binary classification) using features like salary, satisfaction level, and years at company.
- Use Pandas for data ingestion, cleaning, and pre-processing (handling missing values, encoding, scaling).
- Build an ANN using the Keras Sequential API with appropriate input, hidden, and output layers.
- Compile the model with a suitable loss function and optimizer, then train it using `model.fit()`.
- Evaluate the trained model using regression metrics (MAE/RMSE) or classification metrics (Accuracy, Confusion Matrix, F1-Score) as applicable.

Theory:

- Artificial Neural Network (ANN): A computational model inspired by the human brain, made up of interconnected "neurons" arranged in layers, capable of learning complex non-linear patterns from data. Structure of ANN: Consists of an Input Layer (receives features), one or more Hidden Layers (learn patterns), and an Output Layer (produces the prediction).
- Neuron/Perceptron: Basic unit that computes a weighted sum of inputs, adds a bias, and passes the result through an activation function.
- Activation Functions: Introduce non-linearity so the network can learn complex relationships. Common types: ReLU (hidden layers), Sigmoid (binary classification output), Softmax (multi-class output), Linear (regression output).
- Forward Propagation: Process of passing input data through the network layer by layer to generate a prediction.
- Backpropagation: Algorithm that calculates the error at the output and propagates it backward to update weights using gradient descent, so the model learns.
- Loss Function: Measures the difference between predicted and actual values. E.g., Mean Squared Error (MSE) for regression, Binary/Categorical Cross-entropy for classification.
- Optimizer: Algorithm (e.g., SGD, Adam, RMSprop) that adjusts model weights to minimize the loss function.
- Epochs & Batch Size: An epoch is one full pass over the training data; batch size is the number of samples processed before the model's weights are updated.
- Overfitting & Dropout: Overfitting occurs when a model memorizes training data instead of generalizing; Dropout randomly disables neurons during training to reduce overfitting.

Basic Code Info (Sample Skeleton using TensorFlow/Keras):

A generalized ANN pipeline that can be adapted for Student Performance, House Price, or Employee Attrition prediction:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 1. Data Ingestion
df = pd.read_csv('dataset.csv')
df = df.dropna() # handle missing values

# 2. Feature/Target split
X = df.drop('target', axis=1)
y = df['target']

# 3. Train-test split & scaling
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Build the ANN
```

```

model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'), Dense(1, activation='linear') # 'sigmoid' for binary classification
])

# 5. Compile the model
model.compile(optimizer='adam',
              loss='mse', # 'binary_crossentropy' for classification
              metrics=['mae']) # 'accuracy' for classification

# 6. Train the model
history = model.fit(X_train, y_train, epochs=50,
                   batch_size=16, validation_split=0.2)

# 7. Evaluate the model
results = model.evaluate(X_test, y_test)
print('Test Loss, Test MAE:', results)

```

Note: For Employee Attrition (classification), change the output layer to Dense(1, activation='sigmoid'), the loss to 'binary_crossentropy', and evaluate using accuracy, confusion_matrix, and classification_report from sklearn.metrics.

Program: House Price Prediction using ANN (Housing.csv)

The following is the complete, ready-to-run implementation of the House Price Prediction case study, built on the actual dataset used for this practical (Housing.csv), which contains the columns: price (target), area, bedrooms, bathrooms, stories, mainroad, guestroom, basement, hotwaterheating, airconditioning, parking, prefarea, and furnishingstatus.

Step 1: Install/Check Requirements

```

pip install pandas numpy scikit-learn tensorflow

```

Step 2: Import Libraries

```

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

```

Step 3: Load Dataset

```

df = pd.read_csv("Housing.csv")

# View dataset

```

```
df.head()

# Check information
df.info()

# Check missing values
df.isnull().sum()
```

Step 4: Understand Dataset

```
df.columns
```

Housing.csv contains the following columns:

```
price, area, bedrooms, bathrooms, stories, mainroad, guestroom,
basement, hotwaterheating, airconditioning, parking, prefarea,
furnishingstatus
```

Here,

Target Variable: price

Features: area, bedrooms, bathrooms, stories, mainroad, guestroom, basement, hotwaterheating, airconditioning, parking, prefarea, furnishingstatus

Step 5: Handle Missing Values

```
# If any values are missing
df = df.dropna()

# Verify
df.isnull().sum()
```

Step 6: Encode Categorical Columns

The columns mainroad, guestroom, basement, hotwaterheating, airconditioning and prefarea contain 'yes'/'no' text, and furnishingstatus contains three text categories (furnished, semi-furnished, unfurnished). Convert them into numbers using one-hot encoding.

```
df = pd.get_dummies(df, columns=[
    'mainroad', 'guestroom', 'basement', 'hotwaterheating',
    'airconditioning', 'prefarea', 'furnishingstatus'
], drop_first=True)

# Now view dataset
df.head()
```

Step 7: Feature and Target Split

```
X = df.drop("price", axis=1)

y = df["price"]
```

Step 8: Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Step 9: Feature Scaling

ANN performs much better after scaling.

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

Step 10: Build ANN Model

```
model = Sequential()

model.add(Dense(64,
                activation='relu',
                input_shape=(X_train.shape[1],)))

model.add(Dense(32,
                activation='relu'))

model.add(Dense(16,
                activation='relu'))

model.add(Dense(1,
                activation='linear'))
```

Explanation:

```
Input Layer
|
v
64 neurons
|
v
32 neurons
|
v
16 neurons
|
v
Output Layer (Price)
```

Step 11: Compile Model

For Regression -> Loss Function: Mean Squared Error, Optimizer: Adam

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

Step 12: Train Model

```
history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=16
)
```

Training output will show, for every epoch: loss, mae, val_loss, val_mae.

Step 13: Evaluate Model

```
loss, mae = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test MAE:", mae)
```

Step 14: Predict House Prices

```
predictions = model.predict(X_test)

print(predictions[:5])
```

Step 15: Calculate RMSE

```
rmse = np.sqrt(mean_squared_error(y_test, predictions))

print("RMSE :", rmse)
```

Step 16: Compare Actual vs Predicted

```
comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": predictions.flatten()
})

comparison.head(10)
```

Step 17: Save Model (Optional)

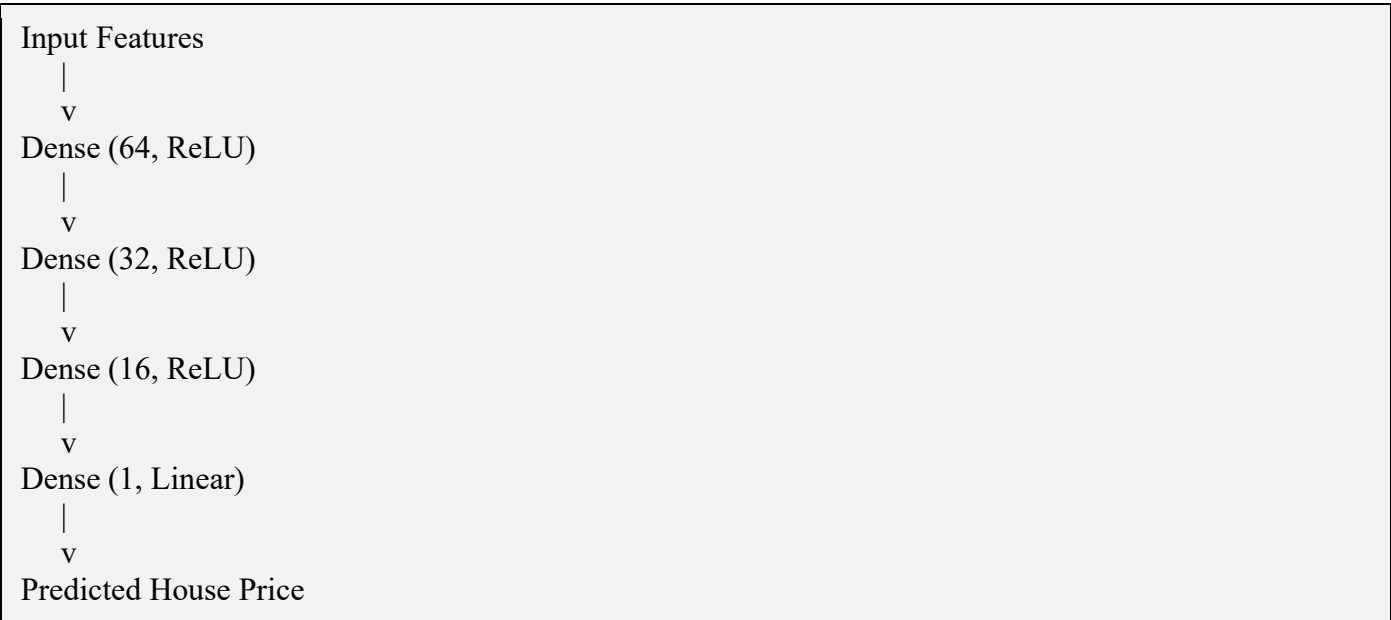
```
model.save("HousePriceANN.keras")
```

Step 18: Observe Results

Record the Test Loss, MAE, and RMSE after running the notebook. Example (your values will differ based on the run):

Test Loss : 2548912
MAE : 1325
RMSE : 1596

ANN Architecture Diagram



Why These Choices?

Component	Used	Reason
Problem Type	Regression	Predicting continuous house prices
Output Layer	Dense(1, activation='linear')	Continuous numerical output
Hidden Activation	ReLU	Efficient and widely used
Loss Function	Mean Squared Error (MSE)	Standard loss for regression
Optimizer	Adam	Fast convergence and adaptive learning
Evaluation Metrics	MAE, RMSE	Measures prediction error

Frequently Asked Questions:

Q: What is an Artificial Neural Network, and how does it mimic biological neurons?

Ans: An ANN is a computational model made of layers of interconnected nodes (neurons). Each artificial neuron receives inputs, multiplies them by weights, adds a bias, and passes the result through an activation function - loosely mimicking how a biological neuron integrates incoming signals and 'fires' once a threshold is crossed.

Q: What is the role of activation functions in an ANN? Explain ReLU, Sigmoid, and Softmax.

Ans: Activation functions introduce non-linearity, allowing the network to learn complex, non-linear relationships rather than acting as a simple linear function. ReLU (max(0, x)) is used in hidden layers because it is computationally efficient and reduces the vanishing-gradient problem. Sigmoid squashes

output into the range (0, 1) and is used for binary classification outputs. Softmax converts the outputs of the final layer into a probability distribution across classes and is used for multi-class classification.

Q: Explain the difference between forward propagation and backpropagation.

Ans: Forward propagation is the process of passing input data through the network, layer by layer, to compute the predicted output. Backpropagation is the process that follows: the error between the predicted and actual output is calculated using the loss function, and this error is propagated backward through the network to compute gradients, which the optimizer uses to update the weights via gradient descent.

Q: How does the choice of output layer activation differ for regression vs. classification problems?

Ans: For regression, the output layer uses a Linear activation (or no activation) since the target is a continuous, unbounded value. For binary classification, a Sigmoid activation is used to output a probability between 0 and 1. For multi-class classification, a Softmax activation is used to output a probability distribution over all classes.

Q: What are loss functions and optimizers, and how do you choose the right one?

Ans: A loss function measures the difference between the predicted and actual values and guides training by quantifying how wrong the model's predictions are; MSE is common for regression, and Binary/Categorical Cross-entropy for classification. An optimizer (e.g. SGD, Adam, RMSprop) is the algorithm that updates the model's weights to minimize this loss. The choice depends on the problem type - the loss function is chosen based on regression vs classification, while Adam is generally a strong default optimizer because it combines momentum with adaptive learning rates for faster, more stable convergence.

Q: How do epochs, batch size, and learning rate affect model training?

Ans: An epoch is one complete pass over the entire training dataset; more epochs let the model learn more but can lead to overfitting if too many are used. Batch size is the number of samples processed before the weights are updated; smaller batches give noisier but more frequent updates, while larger batches give smoother but computationally heavier updates. The learning rate controls the size of each weight-update step; too high a learning rate can cause the model to diverge or overshoot, while too low a value makes training very slow.

Q: What is overfitting in ANN, and how does Dropout help prevent it?

Ans: Overfitting occurs when a model learns the training data too well, including its noise, and as a result performs poorly on unseen/test data. Dropout combats this by randomly disabling (setting to zero) a fraction of neurons during each training step, which prevents the network from relying too heavily on any specific neuron and encourages it to learn more generalizable patterns.

Q: How do you evaluate ANN model performance differently for regression and classification tasks?

Ans: For regression tasks, performance is evaluated using error-based metrics such as MAE (Mean Absolute Error), MSE, and RMSE (Root Mean Squared Error), which measure how far predictions are from actual continuous values. For classification tasks, performance is evaluated using metrics such as Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix, which measure how correctly the model assigns discrete class labels.

Viva Questions

Q: Why Linear activation in output layer?

Ans: Because house price is a continuous value.

Q: Why ReLU in hidden layers?

Ans: It introduces non-linearity and helps the model learn complex patterns while avoiding vanishing gradients.

Q: Why StandardScaler?

Ans: ANNs train faster and more effectively when input features are normalized to a similar scale.

Q: Why use MSE loss?

Ans: MSE penalizes larger prediction errors more strongly, making it suitable for regression tasks.

Q: Why Adam optimizer?

Ans: Adam combines momentum and adaptive learning rates, leading to faster and more stable convergence.

Q: Why train-test split?

Ans: To evaluate the model on unseen data and estimate its generalization performance.

Q: Difference between MAE and RMSE?

Ans: MAE (Mean Absolute Error) is the average absolute difference between predicted and actual values and is easy to interpret. RMSE (Root Mean Squared Error) gives higher weight to larger errors, making it more sensitive to outliers.

Conclusion:

Thus, we have implemented the assignment based on Artificial Neural Network (ANN) using TensorFlow/Keras for prediction problems, covering Student Performance Prediction, House Price Prediction, and Employee Attrition Prediction.

Text Books:

- Joseph Babcock & Raghav Bali, Generative AI with Python and TensorFlow: Build Intelligent Generative AI Models, Packt Publishing, 2024, ISBN-10: 1807780211, ISBN-13: 9781807780210.
- Chip Huyen, AI Engineering: Building Applications with Foundation Models, O'Reilly Media, 2025, ISBN-10: 1098166302, ISBN-13: 9781098166304.
- Mariprasath Dhanasekaran, Cheepati & Rivera, Practical Guide to Machine Learning, NLP, and Generative AI: Libraries, Algorithms and Applications, River Publishers, 2025, ISBN-13: 9788770046534.
- Aurelien Geron, Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, 3rd Edition, O'Reilly Media, 2022, ISBN-10: 9355421987, ISBN-13: 9789355421982.

Reference Books:

- Lewis Tunstall, Leandro von Werra & Thomas Wolf, Natural Language Processing with Transformers: Building Language Applications with Hugging Face, O'Reilly Media, 2022, ISBN-10: 1098103246, ISBN-13: 9781098103248.
- Aston Zhang, Zachary C. Lipton, Mu Li & Alexander J. Smola, Dive into Deep Learning, Cambridge University Press, 2021, ISBN-10: 1009389432, ISBN-13: 9781009389433.

Prepared By,

Mr. U. B. Sangule

Subject In-charge

Approved By,

Dr. M. A. Jawale

HOD-IT